

Comprehensive Empirical Analysis of Nano-Scale YOLO Detectors in Video Object Tracking

Amr Yasser

May 1, 2026

Abstract

This report presents an exhaustive empirical analysis comparing the inference speed and tracking efficiency of three nano-scale object detection models: YOLOv10n, YOLOv11n, and YOLOv12n. ByteTrack was utilized as the standard association algorithm across all tests to isolate the pure detection cost. The experiment tracks vehicles in a 30 FPS high-definition (720p) video sequence. We provide a detailed breakdown of predict versus track latency, frame-by-frame volatility, cumulative processing times, and detection density. Results indicate that while newer architectures like YOLOv12n offer advanced feature extraction capabilities, they exhibit a measurable increase in inference latency and variance compared to the highly deterministic YOLOv10n baseline.

1 Introduction

Real-time object tracking is a critical component of intelligent transportation systems, surveillance, and autonomous driving. The YOLO (You Only Look Once) family of models has dominated the real-time detection paradigm. With the introduction of YOLOv10n, YOLOv11n, and YOLOv12n, there is a continuous evolution in the trade-off between mean Average Precision (mAP) and computational cost. This experiment benchmarks these three nano-scale models, maintaining a constant tracking algorithm (ByteTrack), precision (FP16), and image resolution (640px) to provide an apples-to-apples comparison of their computational footprint.

Source video, frame 0 (1280x720, 30.00 fps, 942 frames)



Figure 1: Initial frame of the source video sequence (Test.mp4).

2 Experimental Setup

The controlled experiment was executed within a reproducible Kaggle environment. To ensure measurement integrity, 30 warmup frames were discarded before timing commenced, mitigating initial CUDA kernel launch overheads. Synchronization barriers (`torch.cuda.synchronize()`) were strictly enforced.

- **Dataset:** `Test.mp4` (Resolution: 1280x720, 30 FPS, 942 frames, ≈ 31.4 s)
- **Hardware:** Tesla T4 GPU, Torch 2.10.0+cu128
- **Configuration:** `imgsz=640`, `half=True`, `conf=0.25`, `iou=0.45`
- **Target Classes:** Vehicles (COCO classes 2, 3, 5, 7)
- **Tracker:** ByteTrack (`bytetrack.yaml`)

3 Headline Performance Evaluation

The raw performance metrics demonstrate a progressive increase in parameter count and computational complexity from v10n to v12n. The pure detection pass (*Predict*) isolates the network’s forward pass, while the *Track* pass includes detection, ByteTrack association, Kalman filter updates, and bounding box rendering.

Table 1: Aggregate Speed and Detection Metrics

Model	Params	Pred. FPS	Track FPS	Pred. Mean	Track Mean	Track p95	Tracks
YOLOv10n	2,775,520	85.1	64.2	11.78 ms	16.16 ms	17.17 ms	200
YOLOv11n	2,624,080	78.3	60.6	12.81 ms	16.55 ms	17.99 ms	184
YOLOv12n	2,603,056	55.3	47.6	18.10 ms	21.09 ms	22.87 ms	149

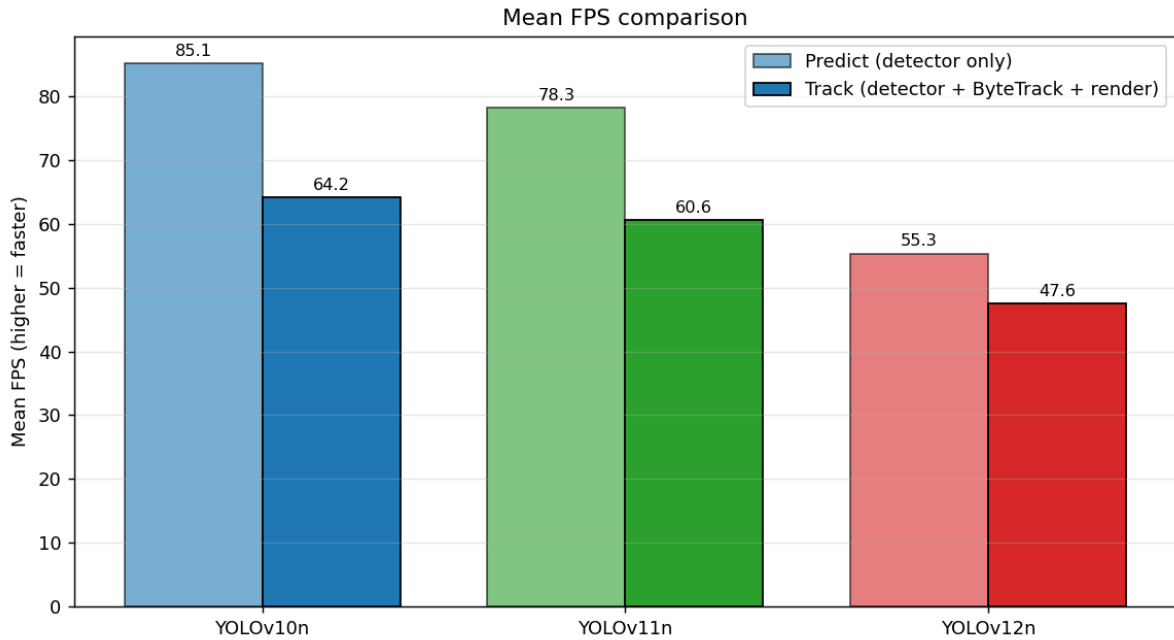


Figure 2: Mean FPS comparison between pure detection and the full tracking pipeline.

4 Detailed Statistical Analysis

4.1 Pipeline Stage Breakdown

The computational footprint can be isolated into preprocessing, forward pass inference, and postprocessing. As illustrated below, the inference phase dominates the latency budget, specifically punishing YOLOv12n.

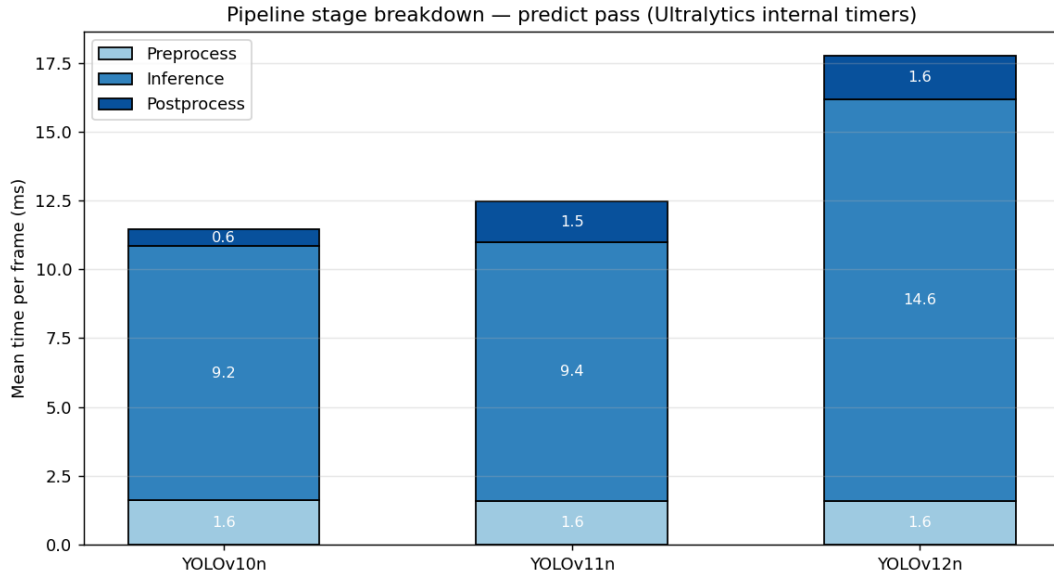


Figure 3: Pipeline stage breakdown isolating the forward pass inference time.

4.2 Frame-by-Frame Latency and Variance

Mean metrics can obscure system jitter. The 95th percentile (p95) tracking latency and overall variance are critical for strict real-time pipelines. YOLOv10n demonstrates exceptional deterministic reliability.

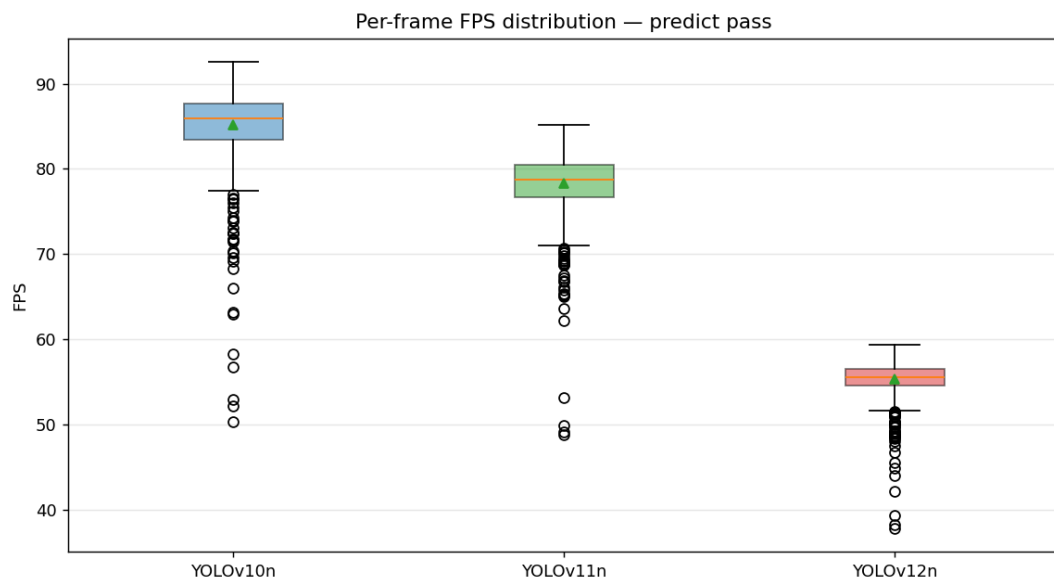


Figure 4: Boxplot demonstrating the FPS variance and distribution during the prediction pass.

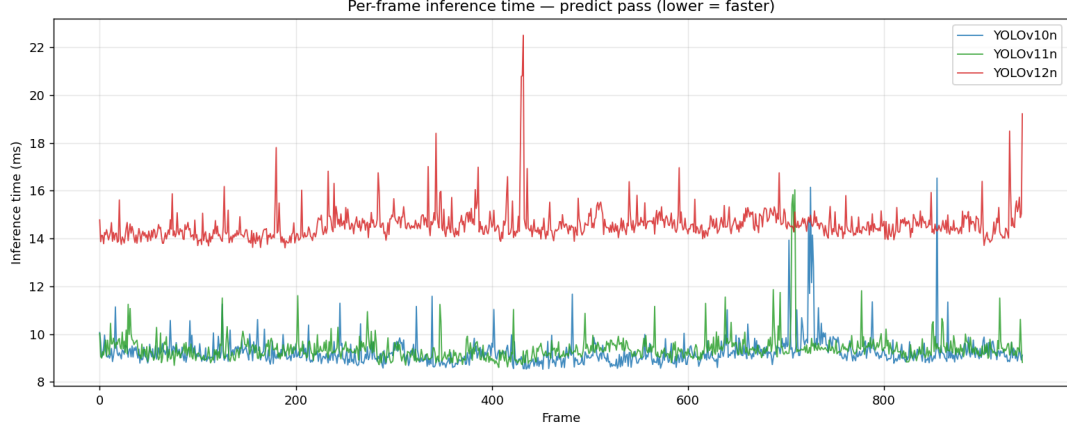


Figure 5: Per-frame inference time during the prediction pass (lower is faster).

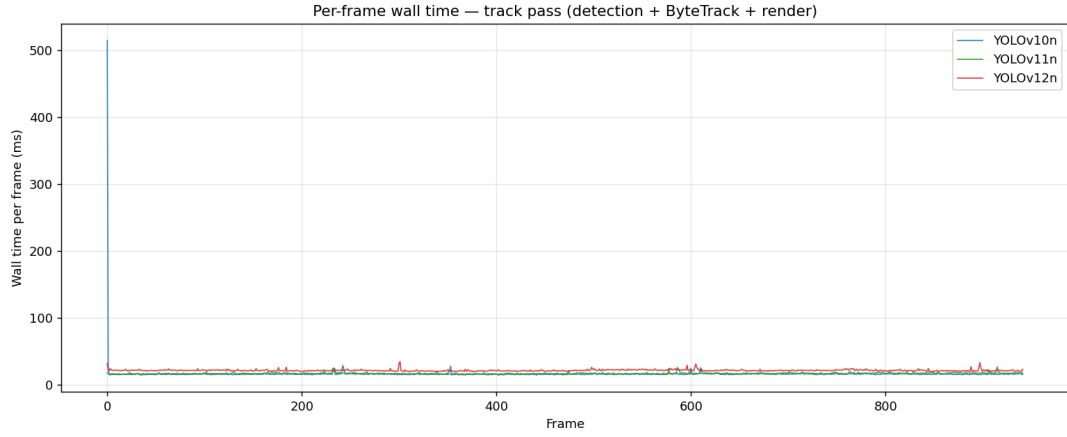


Figure 6: Per-frame wall time during the tracking pass, demonstrating the volatility introduced by the tracking algorithm overhead.

4.3 Cumulative Processing Time

The cumulative processing time chart visualizes the total delay accumulated over the 942-frame sequence. YOLOv12n clearly diverges from the pack.

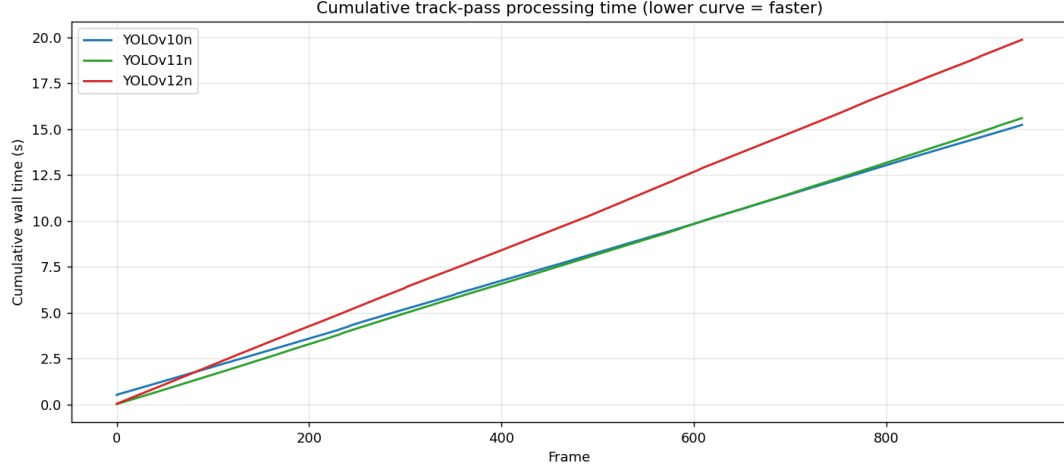


Figure 7: Cumulative tracking pass wall time over the video sequence.

4.4 Detection Density and Consistency

The number of unique tracked identities varies. While this indicates differences in false-positive suppression or identity-switching behavior, sharp divergences in detection density can skew FPS measurements, as fewer active tracks reduce Kalman filter update costs.

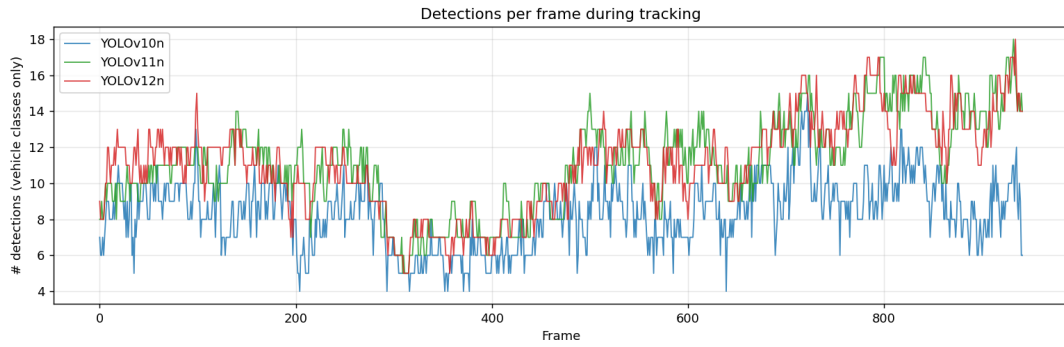


Figure 8: Number of active detections per frame. Note that YOLOv10n consistently maintains higher track counts.

5 Qualitative Tracking Results

To provide qualitative context to the quantitative metrics, mid-video sample frames are extracted.

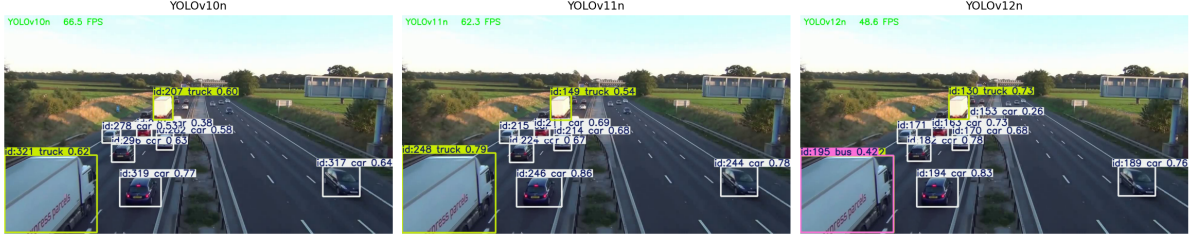
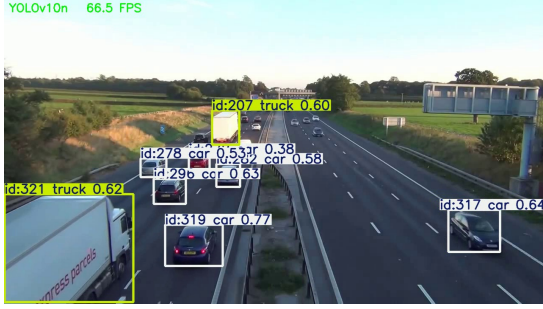
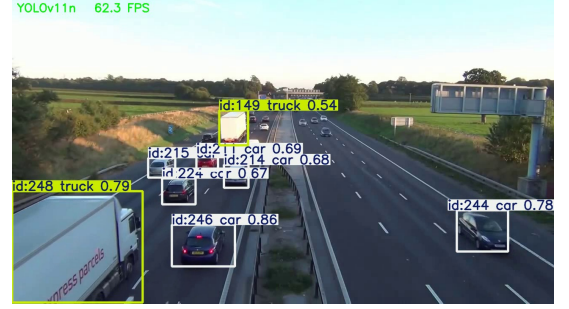


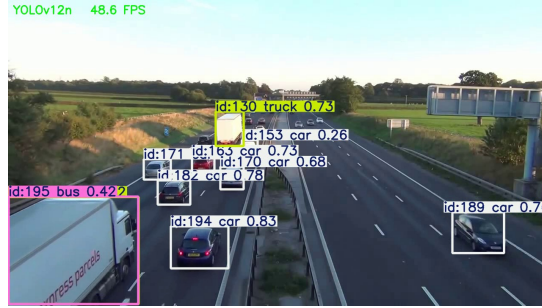
Figure 9: Composite view of sample frames with tracking annotations.



(a) YOLOv10n Sample Frame



(b) YOLOv11n Sample Frame



(c) YOLOv12n Sample Frame

Figure 10: Detailed mid-sequence tracking visualizations for each nano model.

6 Conclusion

For constrained environments requiring maximum throughput, YOLOv10n remains the optimal choice, delivering the highest FPS and lowest latency variance. YOLOv11n offers a competitive middle ground, trading a marginal 8.7% increase in pure detection latency for potentially refined architectural features. YOLOv12n, despite its lower parameter count, incurs a significant 53.6% detection latency penalty, demanding careful consideration if deployed in latency-critical real-time tracking systems.